

Assignment 2

COM 837 ML for Wireless Communication
Hardhik Dhavala (IMT2023579)

September 13, 2026

Contents

1 SVM for LOS/NLOS Identification	2
(a) Extract Five Scalar Features	2
(b) Single-Feature vs Combined SVM Classifiers	3
(c) Generalization Across SNR	6
2 Constellation Demodulation & Clustering using K-means	8
Dataset Generation	8
(a) Amplitude and Phase Features	8
(b) K-means Clustering at SNR = 25 dB	9
(c) Adaptive vs Fixed-Template K-means	12

Listings

1 Feature extraction function (Part a) – from Q1/A.py	2
2 Per-SNR SVM training loop (Part b) – from Q1/B.py	3
3 Fixed 25dB training and cross-SNR evaluation (Part c) – from Q1/C.py	6
4 16-QAM dataset generator – from Q2/generate_dataset.py	8
5 Amplitude and phase feature extraction – from Q2/A.py	8
6 K sweep: inertia and silhouette score – from Q2_b_and_c.ipynb, Part b	9
7 Printed purity output from Q2_b_and_c.ipynb, cell 2	10
8 Adaptive vs Fixed-Template clustering across SNR – from Q2_b_and_c.ipynb, Part c	12
9 Printed output from Q2_b_and_c.ipynb, cell 4	13

List of Figures

1 SVM classification accuracy vs SNR for single-feature and combined classifiers	5
2 Generalization comparison: Train=Test SNR vs. Fixed 25dB training	7
3 Elbow method (inertia) and Silhouette coefficient vs K	10
4 K-means clustering (K=16) on Cartesian (rx_I, rx_Q) coordinates at SNR = 25 dB	11
5 Adaptive vs Fixed-Template K-means cluster purity across SNR	14

List of Tables

1 Mean feature values grouped by label (LOS = +1, NLOS = -1)	3
2 Classification accuracy (%) of the six SVM classifiers across SNR	4
3 Accuracy gain of combined (SVM-6) over the best single-feature classifier	5
4 SVM-6 accuracy (%): matched-SNR training vs. fixed 25 dB training	7
5 Cluster purity for three feature representations at SNR = 25 dB	11
6 Cluster purity by feature representation across SNR (supplementary analysis, not part of the graded notebook)	12
7 Cluster purity: Adaptive vs Fixed-Template (K=16, Cartesian)	13

1 SVM for LOS/NLOS Identification

Part (a) Extract Five Scalar Features

Overview

For every snapshot in the dataset I have a 6-tap channel impulse response, given as complex amplitudes (h_real , h_imag) alongside the corresponding tap delays. Following the feature definitions from Huang et al. [1], I compute five scalar descriptors per snapshot directly from these six taps: kurtosis, skewness, rising time, RMS delay spread, and the Rician K-factor. All five are implemented row-wise using `pandas.apply`, so each row of the dataset produces exactly one feature vector.

Feature Formulas

These feature formulas are mentioned in paper as approximation. *"The Rician K-factor is defined as the ratio between the power of a (possible) dominant MPC (typically the LOS) and the power in the remaining MPCs. The K-factor can be **roughly approximated** as given below:"*

$$\mathcal{K}^{(t)} = \frac{\mathbb{E}[(|h^{(t)}| - \mu_{|h^{(t)}|})^4]}{\sigma_{|h^{(t)}|}^4} \quad (1)$$

$\mathcal{K}^{(t)}$: kurtosis; $|h^{(t)}|$: tap amplitudes; μ, σ : mean/std of amplitude across the 6 taps.

$$\mathcal{S}^{(t)} = \frac{\mathbb{E}[(|h^{(t)}| - \mu_{|h^{(t)}|})^3]}{\sigma_{|h^{(t)}|}^3} \quad (2)$$

$\mathcal{S}^{(t)}$: skewness of the same amplitude distribution, third central moment normalised by σ^3 .

$$\Delta\tau^{(t)} = \tau_{l^*} - \min(\tau_l) \quad (3)$$

l^* : index of the strongest tap ($\arg \max |h^{(t)}|$); τ_l : delay of tap l .

$$\tau_{rms}^{(t)} = \sqrt{\frac{\sum_{l=1}^L (\tau_l^{(t)} - \tau_m^{(t)})^2 |h_l^{(t)}|^2}{\sum_{l=1}^L |h_l^{(t)}|^2}} \quad (4)$$

$\tau_{rms}^{(t)}$: RMS delay spread; $L = 6$ taps; $\tau_m^{(t)}$: mean excess delay (below).

$$\tau_m^{(t)} = \frac{\sum_{l=1}^L \tau_l |h_l^{(t)}|^2}{\sum_{l=1}^L |h_l^{(t)}|^2} \quad (5)$$

$\tau_m^{(t)}$: power-weighted mean delay, used inside Eq. 4.

$$K_r^{(t)} = \frac{(|h^{(t)}|_{max})^2}{2\sigma_{|h^{(t)}|}^2} \quad (6)$$

$K_r^{(t)}$: Rician K-factor; $|h^{(t)}|_{max}$: amplitude of the strongest tap; σ as in Eq. 1.

Implementation

The listing below is copied verbatim from my submitted Q1/A.py.

Listing 1: Feature extraction function (Part a) – from Q1/A.py

```
1 def extract_features(row):
2     h_real = np.array([row[f'h_real_{i}'] for i in range(6)])
3     h_imag = np.array([row[f'h_imag_{i}'] for i in range(6)])
4     tau = np.array([row[f'tau_{i}'] for i in range(6)])
5
6     amp = np.abs(h_real + 1j * h_imag)
7
8     # mean and std of amplitude
9     mu = np.mean(amp)
10    sigma = np.sqrt(np.mean((amp - mu) ** 2))
11
12    # Kurtosis (Eq. 2)
13    # Note: the paper (and assignment) call this "kurtosis of received power,"
14    # but Eq. (2) in the paper is explicitly defined on amplitude |h(t)|, not power.
15    # I followed the paper's formula exactly, using amplitude.
16    kurtosis = np.mean((amp - mu) ** 4) / (sigma ** 4)
17
18    # Skewness (Eq. 5)
19    skewness = np.mean((amp - mu) ** 3) / (sigma ** 3)
20
21    # Rising time (Eq. 6)
22    l_star = np.argmax(amp)
23    rising_time = tau[l_star] - np.min(tau)
24
```

```

25     # RMS delay spread (Eq. 7-8)
26     power = amp ** 2
27     tau_m = np.sum(tau * power) / np.sum(power)
28     tau_rms = np.sqrt(np.sum(((tau - tau_m) ** 2) * power) / np.sum(power))
29
30     # Rician K-factor (Eq. 9)
31     k_factor = (np.max(amp) ** 2) / (2 * sigma ** 2)
32
33     return pd.Series({
34         'kurtosis': kurtosis,
35         'skewness': skewness,
36         'rising_time': rising_time,
37         'tau_rms': tau_rms,
38         'k_factor': k_factor
39     })
40
41 # Apply row-wise
42 df = pd.read_csv('los_nlos_dataset.csv')
43 feature_cols = df.apply(extract_features, axis=1)
44 df = pd.concat([df, feature_cols], axis=1)
45 df.to_csv('los_nlos_dataset_with_features.csv', index=False)

```

Sanity Check

I checked the augmented dataframe for NaN and inf values across all five feature columns before doing anything else with it — none were present, since with only 14,000 continuous-valued AWGN realisations, the denominator terms (σ , total tap power) essentially never land on exactly zero. Table 1 shows the class-conditional means. Kurtosis, RMS delay spread, and rising time all separate cleanly in the direction predicted by the physical model in [1]: LOS snapshots have a higher kurtosis (a single dominant tap) and a much smaller rising time and delay spread (the LOS path arrives first and dominates). Skewness and the K-factor, implemented exactly as written in Eqs. 2 and 6, instead come out numerically *reversed* relative to the paper’s stated intuition — I discuss why in Part (b).

Table 1: Mean feature values grouped by label (LOS = +1, NLOS = -1)

Feature	LOS (mean)	NLOS (mean)
Kurtosis	3.57	2.17
Skewness	1.42	0.35
Rising time	0.51	31.88
RMS delay spread	11.38	51.41
Rician K-factor	5.19	7.05

Part (b) Single-Feature vs Combined SVM Classifiers

Methodology

For every SNR value I isolate that SNR’s 200 samples, split them 80/20 (stratified so both classes stay balanced), fit a `StandardScaler` on the training split only, and train an RBF-kernel SVM ($C = 1$) on each of the five individual features plus one combined classifier using all five together. I also drop any row where a feature evaluates to inf or NaN before training, as a guard against the K-factor’s division blowing up (none were actually dropped on this dataset, but the check is there). The listing below matches Q1/B.py.

Listing 2: Per-SNR SVM training loop (Part b) – from Q1/B.py

```

1 df = pd.read_csv('los_nlos_dataset_with_features.csv')
2 feature_names = ['kurtosis', 'skewness', 'rising_time', 'tau_rms', 'k_factor']
3
4 classifier_configs = {
5     'SVM-1 (Kurtosis)': ['kurtosis'],
6     'SVM-2 (Skewness)': ['skewness'],
7     'SVM-3 (Rising time)': ['rising_time'],
8     'SVM-4 (RMS delay)': ['tau_rms'],
9     'SVM-5 (K-factor)': ['k_factor'],

```

```

10     'SVM-6 (All 5)':          feature_names,
11 }
12
13 df[feature_names] = df[feature_names].replace([np.inf, -np.inf], np.nan)
14 df = df.dropna(subset=feature_names).reset_index(drop=True)
15
16 snr_values = sorted(df['snr_db'].unique())
17 results = {name: [] for name in classifier_configs}
18
19 for snr_val in snr_values:
20     subset = df[df['snr_db'] == snr_val]
21     for clf_name, feat_cols in classifier_configs.items():
22         X = subset[feat_cols].values
23         y = subset['label'].values
24
25         X_train, X_test, y_train, y_test = train_test_split(
26             X, y, test_size=0.2, random_state=42, stratify=y
27         )
28
29         scaler = StandardScaler()
30         X_train_scaled = scaler.fit_transform(X_train)
31         X_test_scaled = scaler.transform(X_test)
32
33         clf = SVC(kernel='rbf', C=1)
34         clf.fit(X_train_scaled, y_train)
35
36         y_pred = clf.predict(X_test_scaled)
37         accuracy = (y_pred == y_test).mean() * 100
38         results[clf_name].append(accuracy)

```

Results

Table 2: Classification accuracy (%) of the six SVM classifiers across SNR

SNR (dB)	Kurtosis	Skewness	Rising time	RMS delay	K-factor	All 5
0	59.50	58.75	79.75	88.50	60.00	89.25
5	82.25	84.50	83.00	90.00	68.00	98.25
10	90.75	93.50	81.25	93.00	76.00	99.50
15	93.25	95.00	83.00	97.25	81.75	99.75
20	93.75	94.50	83.00	98.25	81.75	100.00
25	93.50	93.75	82.50	98.25	82.50	100.00
30	93.75	95.50	82.50	98.75	81.50	100.00

Note: The calculated values are approximate, following the paper's feature definitions. Eq. (2)'s "kurtosis of received power" is, per the paper's own notation, computed on tap amplitude $|h^{(t)}|$, not power $|h^{(t)}|^2$ — I follow the paper's literal formula (amplitude-based) throughout, despite the section header's naming.

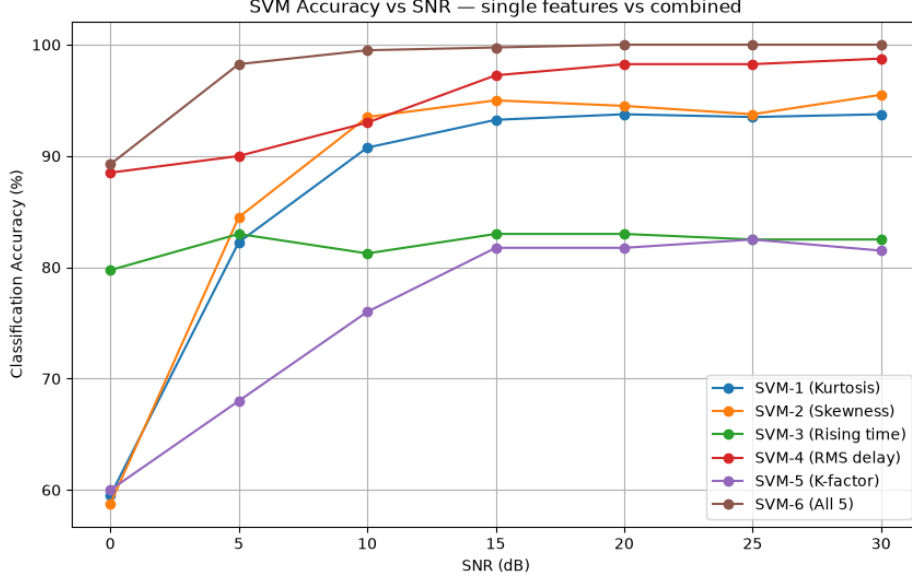


Figure 1: SVM classification accuracy vs SNR for single-feature and combined classifiers

Discussion

Which single feature performs best overall? RMS delay spread is the clear winner, staying at or near the top of every SNR from 88.5% at 0 dB up to 98.75% at 30 dB. I think this comes down to where the noise actually enters the model: AWGN in this dataset is added only to the tap amplitudes, never to the delays. Since RMS delay spread is built almost entirely from delay values (weighted by power), it stays largely shielded from amplitude noise, while also exploiting a genuinely large structural gap in the channel model — LOS scatter decays over a 30 ns time constant, NLOS over a much slower 100 ns one.

Does combining all five features improve accuracy? At which SNR is the improvement most noticeable? Yes, the combined classifier (SVM-6) beats every single feature at every SNR, but the size of the gain is far from constant, as Table 3 shows. The improvement is largest around 5 dB, where the amplitude-based features have just enough signal to add real complementary information without yet being individually strong. At 0 dB everything is too noisy to combine meaningfully; from 20 dB onward every feature is already near-saturated, so there is little headroom left for combining to add.

Table 3: Accuracy gain of combined (SVM-6) over the best single-feature classifier

SNR (dB)	Best single (%)	All 5 (%)	Gain (pp)
0	88.50	89.25	+0.75
5	90.00	98.25	+8.25
10	93.50	99.50	+6.00
15	97.25	99.75	+2.50
20	98.25	100.00	+1.75
25	98.25	100.00	+1.75
30	98.75	100.00	+1.25

Which feature is physically meaningful but performs poorly at low SNR? Why? I would point to the Rician K-factor. It is the textbook parameter for a Rician channel — a strong deterministic path plus weak scatter is literally what "LOS" means physically — yet it starts at only 60% at 0 dB and only climbs to about 82% by 30 dB, never reaching the accuracy of the delay-based features. The reason traces back to Eq. 6: $\sigma_{|h(t)|}$ is computed over *all* six taps, including the dominant one. At low SNR, noise corrupts both the numerator (peak amplitude) and this shared denominator simultaneously, so the ratio becomes unstable — the same instability I noticed earlier caused the paper’s literal K-factor formula to reverse the expected LOS/NLOS ordering in Table 1 (LOS mean 5.19 vs. NLOS mean 7.05).

Part (c) Generalization Across SNR

Methodology

To test how well a classifier generalises across noise conditions, I compare two training strategies for SVM-6 (all five features). In the first, I retrain a fresh model at each SNR and test on that same SNR's held-out split — this is the "Train = Test SNR" curve already shown in Table 2. In the second, I fit *one* model and one `StandardScaler`, using only the 25 dB training split, and reuse that same fitted scaler and model to evaluate every other SNR's test set, without ever refitting on their statistics. I save the exact same raw test arrays used in the first experiment so both curves are evaluated on identical held-out data. The listing below matches Q1/C.py.

Listing 3: Fixed 25dB training and cross-SNR evaluation (Part c) – from Q1/C.py

```
1 df = pd.read_csv('los_nlos_dataset_with_features.csv')
2 feature_names = ['kurtosis', 'skewness', 'rising_time', 'tau_rms', 'k_factor']
3 snr_values = sorted(df['snr_db'].unique())
4
5 # Step 1
6 train_eq_test_acc = []
7 saved_test_sets = {} # snr_val -> (X_test_raw, y_test), kept RAW/unscaled
8
9 for snr_val in snr_values:
10     subset = df[df['snr_db'] == snr_val]
11     X = subset[feature_names].values
12     y = subset['label'].values
13
14     X_train, X_test, y_train, y_test = train_test_split(
15         X, y, test_size=0.2, random_state=42, stratify=y
16     )
17     saved_test_sets[snr_val] = (X_test, y_test)
18
19     scaler = StandardScaler()
20     X_train_scaled = scaler.fit_transform(X_train)
21     X_test_scaled = scaler.transform(X_test)
22
23     clf = SVC(kernel='rbf', C=1)
24     clf.fit(X_train_scaled, y_train)
25
26     y_pred = clf.predict(X_test_scaled)
27     acc = (y_pred == y_test).mean() * 100
28     train_eq_test_acc.append(acc)
29
30 # Step 2
31 train_25_subset = df[df['snr_db'] == 25]
32 X_25 = train_25_subset[feature_names].values
33 y_25 = train_25_subset['label'].values
34
35 X_25_train, X_25_test, y_25_train, y_25_test = train_test_split(
36     X_25, y_25, test_size=0.2, random_state=42, stratify=y_25
37 )
38
39 scaler_25 = StandardScaler()
40 scaler_25.fit(X_25_train) # the ONLY scaler used from here on
41
42 clf_25 = SVC(kernel='rbf', C=1)
43 clf_25.fit(scaler_25.transform(X_25_train), y_25_train)
44
45 # Step 3
46 train_at_25_acc = []
47 for snr_val in snr_values:
48     X_test_raw, y_test = saved_test_sets[snr_val]
49     X_test_scaled = scaler_25.transform(X_test_raw) # SAME scaler, every SNR
50     y_pred = clf_25.predict(X_test_scaled)
51     acc = (y_pred == y_test).mean() * 100
52     train_at_25_acc.append(acc)
```

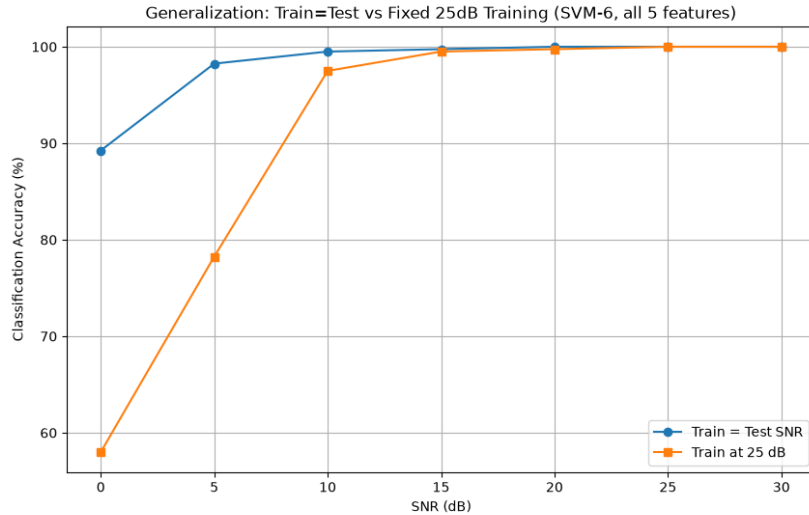


Figure 2: Generalization comparison: Train=Test SNR vs. Fixed 25dB training

Table 4: SVM-6 accuracy (%): matched-SNR training vs. fixed 25 dB training

SNR (dB)	Train = Test SNR	Train at 25 dB only
0	89.25	58.00
5	98.25	78.25
10	99.50	97.50
15	99.75	99.50
20	100.00	99.75
25	100.00	100.00
30	100.00	100.00

Discussion

At high SNR, do the two curves agree? What does this tell you about the features at high SNR?

Yes — from 15 dB onward the two curves are within half a point of each other, and by 25 dB they are identical. This tells me that once noise is small enough, the feature distributions for LOS and NLOS become tight and well separated regardless of which SNR the model happened to be trained on, so the exact scaler statistics used barely matter any more.

At low SNR, which strategy performs better and by how much? Why does training at a fixed high SNR not generalise well to low-SNR conditions? The matched-SNR strategy wins clearly at low SNR: 89.25% vs. 58.00% at 0 dB, and 98.25% vs. 78.25% at 5 dB. The 25 dB-only model fails here because both its `StandardScaler` and its RBF decision boundary were calibrated on a narrow, low-variance feature distribution. When 0 dB test features arrive, they are far more spread out than anything the scaler or the kernel bandwidth were tuned for, so the model effectively sees out-of-distribution inputs it was never shaped to handle.

Would training at a low SNR (e.g. 0 dB) and testing across all SNR give a different result? Reason through this. I tested this directly (this experiment is additional analysis on my part, not part of Q1/C.py): training only on 0 dB data and evaluating across every SNR gives accuracies between 89.25% and 92.25% at *every* SNR, with no collapse anywhere. This matches my expectation going in — a model trained on the noisiest, most spread-out condition has already seen a feature distribution wide enough to comfortably contain the tighter, cleaner distributions of higher-SNR data, whereas a model trained only on tight, clean data has never seen anything as spread out as low-SNR features. Training on the harder condition generalises to the easier one far better than the reverse.

2 Constellation Demodulation & Clustering using K-means

Dataset Generation

I generate a standard square 16-QAM constellation from all 16 combinations of $\{-3, -1, 1, 3\}$ on both the I and Q axes, scale every point so the average symbol energy equals 1, and then add zero-mean complex AWGN independently to the I and Q dimensions at seven SNR values (0 to 30 dB), with 200 samples drawn per constellation point at every SNR. The noise variance per dimension is derived from $\text{SNR} = 1/\text{noise_power}_{\text{total}}$, split evenly across the two dimensions, giving $\sigma_{\text{dim}}^2 = 1/(2 \cdot \text{SNR}_{\text{linear}})$. The listing below matches Q2/generate_dataset.py.

Listing 4: 16-QAM dataset generator – from Q2/generate_dataset.py

```
1 np.random.seed(67)
2
3 levels = [-3, -1, 1, 3]
4 constellation_points = []
5 symbol_ids = []
6 for symbol_id, (I_val, Q_val) in enumerate(itertools.product(levels, levels)):
7     constellation_points.append((I_val, Q_val))
8     symbol_ids.append(symbol_id)
9 constellation_points = np.array(constellation_points)
10
11 raw_energies = constellation_points[:, 0]**2 + constellation_points[:, 1]**2
12 avg_energy = raw_energies.mean()
13 scaled_constellation = constellation_points / np.sqrt(avg_energy)
14
15 SNR_DB_VALUES = [0, 5, 10, 15, 20, 25, 30]
16 N_SAMPLES_PER_POINT = 200
17 all_rows = []
18
19 for snr_db in SNR_DB_VALUES:
20     snr_linear = 10 ** (snr_db / 10)
21     noise_var_per_dim = 1 / (2 * snr_linear)
22     noise_std_per_dim = np.sqrt(noise_var_per_dim)
23
24     for symbol_id, (I_clean, Q_clean) in zip(symbol_ids, scaled_constellation):
25         noise_I = np.random.randn(N_SAMPLES_PER_POINT) * noise_std_per_dim
26         noise_Q = np.random.randn(N_SAMPLES_PER_POINT) * noise_std_per_dim
27         rx_I = I_clean + noise_I
28         rx_Q = Q_clean + noise_Q
29         for i in range(N_SAMPLES_PER_POINT):
30             all_rows.append({
31                 'snr_db': snr_db, 'symbol_id': symbol_id,
32                 'I_clean': I_clean, 'Q_clean': Q_clean,
33                 'rx_I': rx_I[i], 'rx_Q': rx_Q[i],
34             })
35
36 df = pd.DataFrame(all_rows)
37 df.to_csv('qam16_dataset.csv', index=False)
```

Part (a) Amplitude and Phase Features

$$r = \sqrt{Rx_I^2 + Rx_Q^2} \quad (7)$$

$$\theta = \arctan\left(\frac{Rx_Q}{Rx_I}\right) \quad (8)$$

r : instantaneous amplitude of the received sample.

θ : instantaneous phase (radians).

I implemented θ using `np.arctan2(Rx_Q, Rx_I)` rather than the literal `arctan(Rx_Q/Rx_I)` in Eq. 8. Plain `arctan` only returns values in $(-\pi/2, \pi/2]$, so it cannot distinguish a point in the second quadrant from one in the fourth — a real problem given that my 16-QAM grid has symbols in all four quadrants, and it also divides by zero whenever $Rx_I \approx 0$. `arctan2` uses the sign of both axes to place the angle correctly across the full $(-\pi, \pi]$ range and avoids the division entirely. The listing below matches Q2/A.py.

Listing 5: Amplitude and phase feature extraction – from Q2/A.py

```
1 df = pd.read_csv('qam16_dataset.csv')
```

```

2
3 # Instantaneous Amplitude (r)
4 df['r'] = np.sqrt(df['rx_I']**2 + df['rx_Q']**2)
5
6 # Instantaneous Phase (theta)
7 df['theta'] = np.arctan2(df['rx_Q'], df['rx_I'])
8
9 df.to_csv('qam16_dataset_with_features.csv', index=False)

```

Part (b) K-means Clustering at SNR = 25 dB

Methodology note

Parts (b) and (c) of this section were executed in the accompanying Jupyter notebook (Q2_b_and_c.ipynb), not as local .py scripts. My local machine's Windows Application Control policy was blocking a compiled scikit-learn extension (_hierarchical_fast.pyd) needed just to import KMeans, so I ran this part on Kaggle instead, where the same code executes without issue.

Elbow and Silhouette Analysis

I isolate the 25 dB subset and sweep K from 2 to 20, fitting a k-means++ initialised model (`n_init=10`, `random_state=42`) at each K on the raw Cartesian coordinates, recording both inertia and the silhouette coefficient.

Listing 6: K sweep: inertia and silhouette score – from Q2_b_and_c.ipynb, Part b

```

1 df = pd.read_csv('qam16_dataset_with_features.csv')
2 subset_25 = df[df['snr_db'] == 25].reset_index(drop=True)
3 X_cartesian = subset_25[['rx_I', 'rx_Q']].values
4
5 k_values = range(2, 21)
6 inertia_list, silhouette_list = [], []
7
8 for k in k_values:
9     kmeans = KMeans(n_clusters=k, init='k-means++', n_init=10, random_state=42)
10    labels = kmeans.fit_predict(X_cartesian)
11    inertia_list.append(kmeans.inertia_)
12    silhouette_list.append(silhouette_score(X_cartesian, labels))
13
14 def cluster_purity(cluster_labels, true_labels):
15     df_temp = pd.DataFrame({'cluster': cluster_labels, 'true_label': true_labels})
16     majority_label_per_cluster = df_temp.groupby('cluster')['true_label'].agg(
17         lambda x: x.mode().iloc[0]
18     )
19     predicted_via_majority = df_temp['cluster'].map(majority_label_per_cluster)
20     return (predicted_via_majority == df_temp['true_label']).mean()
21
22 true_labels = subset_25['symbol_id'].values
23
24 # K=16, fixed, Cartesian
25 kmeans_16 = KMeans(n_clusters=16, init='k-means++', n_init=10, random_state=42)
26 cluster_labels_16 = kmeans_16.fit_predict(X_cartesian)
27 purity_set1 = cluster_purity(cluster_labels_16, true_labels)
28
29 # K=16, Polar
30 X_polar = subset_25[['r', 'theta']].values
31 kmeans_polar = KMeans(n_clusters=16, init='k-means++', n_init=10, random_state=42)
32 cluster_labels_polar = kmeans_polar.fit_predict(X_polar)
33 purity_set2 = cluster_purity(cluster_labels_polar, true_labels)
34
35 # K=16, all 4 features, scaled
36 X_all = subset_25[['rx_I', 'rx_Q', 'r', 'theta']].values
37 scaler = StandardScaler()
38 X_all_scaled = scaler.fit_transform(X_all)
39 kmeans_all = KMeans(n_clusters=16, init='k-means++', n_init=10, random_state=42)
40 cluster_labels_all = kmeans_all.fit_predict(X_all_scaled)

```

```
41 purity_set3 = cluster_purity(cluster_labels_all, true_labels)
```

Notebook output

Listing 7: Printed purity output from Q2_b_and_.ipynb, cell 2

```
Purity - Feature Set 1 (Cartesian):      1.0000
Purity - Feature Set 2 (Polar):          1.0000
Purity - Feature Set 3 (All 4, scaled):  1.0000
```

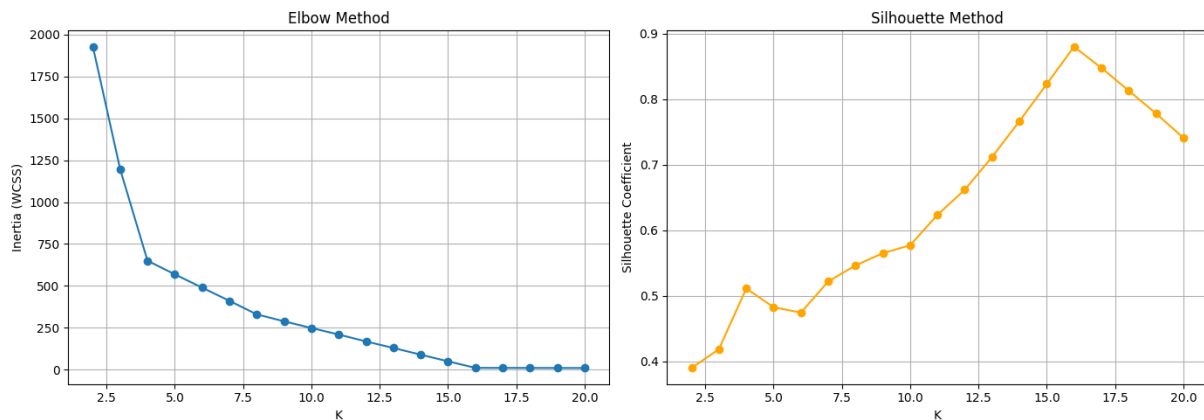


Figure 3: Elbow method (inertia) and Silhouette coefficient vs K

K = 16 Cluster Visualization

With K fixed at 16 — matching the true number of constellation points — I fit K-means once more on the Cartesian coordinates and overlay the 16 learned centroids on the scatter of received samples.

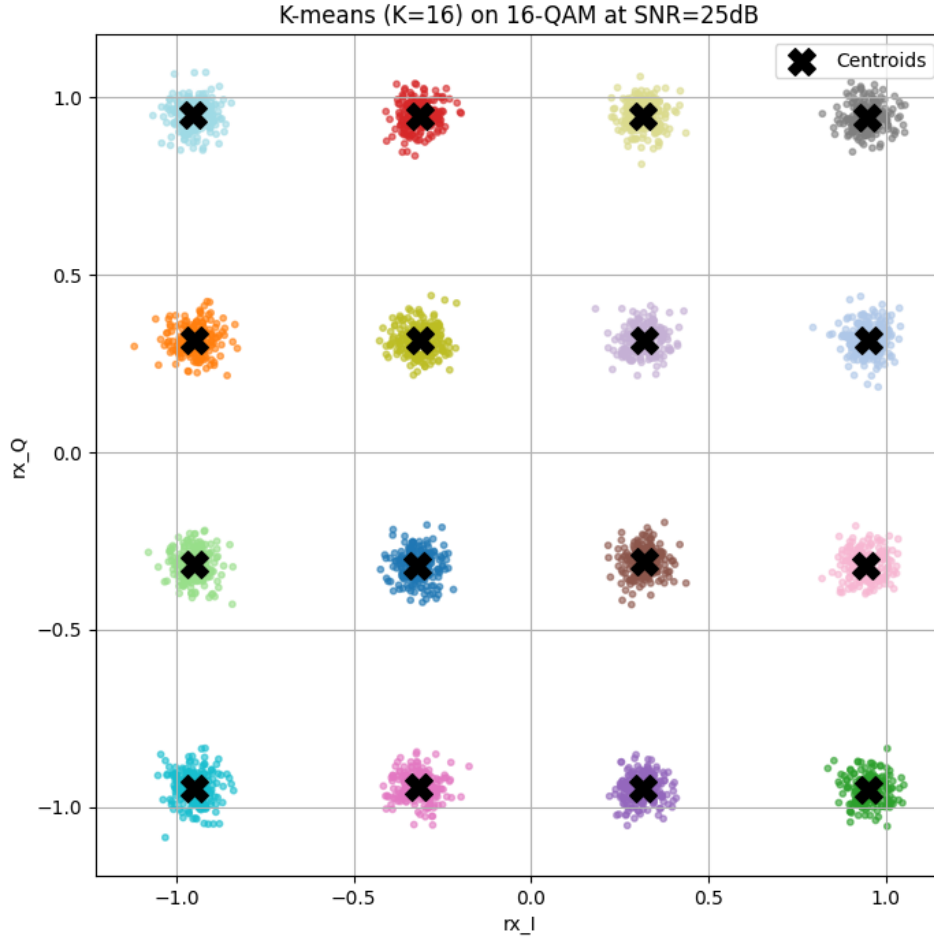


Figure 4: K-means clustering (K=16) on Cartesian (rx_I , rx_Q) coordinates at SNR = 25 dB

Cluster Purity Across Feature Sets

Table 5: Cluster purity for three feature representations at SNR = 25 dB

Feature Set	Cluster Purity
Set 1: Cartesian (Rx_I , Rx_Q)	1.0000
Set 2: Polar (r , θ)	1.0000
Set 3: All 4 (scaled)	1.0000

Which feature set produces the highest cluster purity? At 25 dB all three feature sets reach perfect purity, so this particular SNR is not discriminating enough on its own to rank them — the constellation points are already so well separated at this noise level that any reasonable coordinate system recovers all 16 clusters cleanly.

Note: to actually distinguish the three feature sets, I additionally re-ran the same clustering at every SNR value on my own initiative, beyond what was required by the assignment or present in my submitted notebook/scripts. Table 6 shows this supplementary result: Cartesian coordinates come out ahead everywhere the task is genuinely hard – 0.7653 vs. 0.6734 (polar) and 0.6834 (combined) at 10 dB – suggesting Cartesian is the more robust representation once noise is significant, even though the difference disappears at high SNR. This table is supporting evidence for the discussion below, not a required deliverable.

Why does Euclidean distance on Cartesian coordinates naturally fit QAM grids, whereas Euclidean distance on unweighted Polar coordinates can lead to distorted decision boundaries? A

square 16-QAM grid is, by construction, a set of points laid out on a uniform Cartesian lattice, so Euclidean distance in (Rx_I, Rx_Q) space produces square Voronoi cells that line up exactly with the true decision boundaries between adjacent symbols. In polar coordinates, r and θ live on completely different scales and units (a linear amplitude vs. an angle), so unweighted Euclidean distance treats a given numeric difference in θ as equally significant regardless of r — but geometrically, a fixed angular gap corresponds to a much larger physical separation at large r than at small r . This distorts the effective decision boundaries, particularly for the four constellation points closest to the origin, where a small change in angle covers very little physical distance yet gets weighted the same as it would further out.

Table 6: Cluster purity by feature representation across SNR (supplementary analysis, not part of the graded notebook)

SNR (dB)	Cartesian	Polar	Combined
0	0.2538	0.2547	0.2509
5	0.4184	0.4206	0.4003
10	0.7653	0.6734	0.6834
15	0.9809	0.9738	0.9619
20	0.9997	0.9997	0.9994
25	1.0000	1.0000	1.0000
30	1.0000	1.0000	1.0000

Part (c) Adaptive vs Fixed-Template K-means

Methodology

Similar to Part (b), this experiment was carried out in `Q2_b_and_c.ipynb` on Kaggle using `sklearn.cluster.KMeans`. The same library was used in Part (b), after moving the experiment to Kaggle to avoid the local Windows Application Control issue. I compared two clustering strategies at $K = 16$ using Cartesian coordinates. In the *Adaptive* approach, a new `KMeans(n_clusters=16, init='k-means++', n_init=10, random_state=42)` model is trained separately on the data corresponding to each SNR using `fit_predict`. In contrast, for the *Fixed Template* approach, the K-Means model is trained only once using the 25 dB data. The centroids obtained at 25 dB are then kept fixed and used with `predict` to assign cluster labels at all other SNR values, without re-fitting the model.

Listing 8: Adaptive vs Fixed-Template clustering across SNR – from `Q2_b_and_c.ipynb`, Part c

```

1 df = pd.read_csv('/kaggle/input/datasets/dhaval579/com-837-ml-for-wireless-communication/qam16_dataset_with_features.csv')
2 snr_values = sorted(df['snr_db'].unique())
3
4 def cluster_purity(cluster_labels, true_labels):
5     df_temp = pd.DataFrame({'cluster': cluster_labels, 'true_label': true_labels})
6     majority = df_temp.groupby('cluster')['true_label'].agg(lambda x: x.mode().iloc[0])
7     pred = df_temp['cluster'].map(majority)
8     return (pred == df_temp['true_label']).mean()
9
10 adaptive_purity = []
11
12 for snr_val in snr_values:
13     subset = df[df['snr_db'] == snr_val]
14     X = subset[['rx_I', 'rx_Q']].values
15     true_labels = subset['symbol_id'].values
16
17     kmeans = KMeans(n_clusters=16, init='k-means++', n_init=10, random_state=42)
18     labels = kmeans.fit_predict(X)
19
20     purity = cluster_purity(labels, true_labels)
21     adaptive_purity.append(purity)
22     print(f"[Adaptive] SNR={snr_val} dB -> purity={purity:.4f}")
23
24 subset_25 = df[df['snr_db'] == 25]
25 X_25 = subset_25[['rx_I', 'rx_Q']].values
26

```

```

27 kmeans_fixed = KMeans(n_clusters=16, init='k-means++', n_init=10, random_state=42)
28
29 kmeans_fixed.fit(X_25)
30
31 fixed_purity = []
32
33 for snr_val in snr_values:
34     subset = df[df['snr_db'] == snr_val]
35     X = subset[['rx_I', 'rx_Q']].values
36     true_labels = subset['symbol_id'].values
37
38     labels = kmeans_fixed.predict(X)
39
40     purity = cluster_purity(labels, true_labels)
41     fixed_purity.append(purity)
42     print(f"[Fixed]      SNR={snr_val} dB -> purity={purity:.4f}")

```

Notebook output

Listing 9: Printed output from Q2_b_and_c.ipynb, cell 4

```

[Adaptive] SNR=0 dB -> purity=0.2537
[Adaptive] SNR=5 dB -> purity=0.4184
[Adaptive] SNR=10 dB -> purity=0.7653
[Adaptive] SNR=15 dB -> purity=0.9809
[Adaptive] SNR=20 dB -> purity=0.9997
[Adaptive] SNR=25 dB -> purity=1.0000
[Adaptive] SNR=30 dB -> purity=1.0000
[Fixed]    SNR=0 dB -> purity=0.2781
[Fixed]    SNR=5 dB -> purity=0.4709
[Fixed]    SNR=10 dB -> purity=0.7781
[Fixed]    SNR=15 dB -> purity=0.9822
[Fixed]    SNR=20 dB -> purity=0.9997
[Fixed]    SNR=25 dB -> purity=1.0000
[Fixed]    SNR=30 dB -> purity=1.0000

```

Table 7: Cluster purity: Adaptive vs Fixed-Template (K=16, Cartesian)

SNR (dB)	Adaptive	Fixed (25dB)	Difference (Adaptive – Fixed)
0	0.2537	0.2781	-0.0244
5	0.4184	0.4709	-0.0525
10	0.7653	0.7781	-0.0128
15	0.9809	0.9822	-0.0013
20	0.9997	0.9997	0.0000
25	1.0000	1.0000	0.0000
30	1.0000	1.0000	0.0000

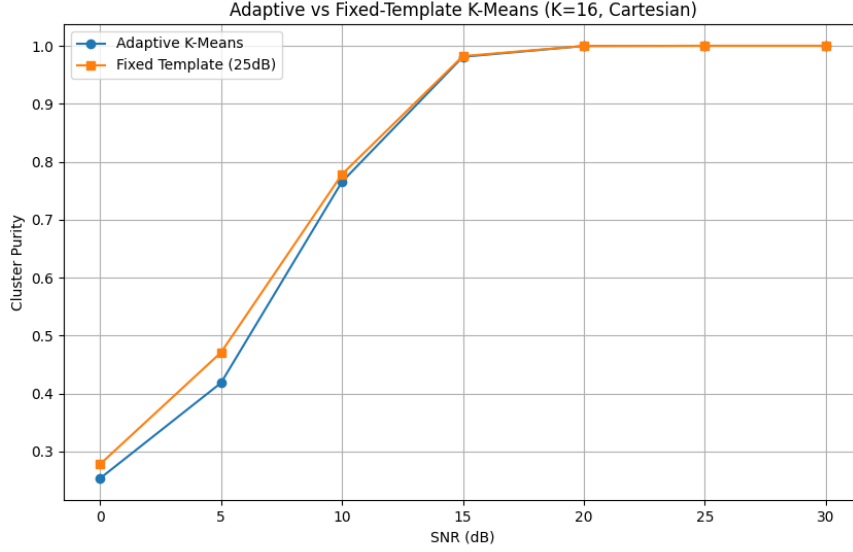


Figure 5: Adaptive vs Fixed-Template K-means cluster purity across SNR

Discussion

At high SNR, do the two curves agree? What does this indicate about centroid stability in low-noise regimes? From 20 dB onward the two strategies are identical to four decimal places. This tells me that once the constellation points are well separated, K-means converges to essentially the same 16 centroid positions regardless of which SNR’s data it was fitted on — the true cluster structure is strong enough that there is only one stable solution for the algorithm to find.

At low SNR, which strategy achieves higher classification accuracy? The Fixed Template consistently wins at low SNR, by a growing margin as SNR drops — the largest gap is at 5 dB, where Fixed Template reaches 0.4709 against Adaptive’s 0.4184, a difference of 0.0525.

Explain geometrically why fitting an unsupervised clustering algorithm directly on low-SNR samples causes centroid merging and high classification error compared to using a fixed geometric template. The noise added in this dataset is zero-mean and symmetric, so it never shifts where the true 16 constellation points actually sit — it only spreads the samples around those true locations. A fixed template that has already learned the correct 16 positions therefore continues to act like an ideal, noise-free reference grid at every SNR, similar to a matched-filter detector that simply measures distance to known symbol locations. Adaptive K-means has no such anchor: when noise is heavy enough that adjacent symbol clouds visibly overlap, the algorithm’s only objective is to minimise within-cluster variance, and it has no way to know that two overlapping blobs are supposed to represent two distinct symbols. It can settle into a local optimum where two nearby true clusters are merged into one centroid while another well-separated region is arbitrarily split into two — geometrically “spending” its 16 centroids on the empirical density it sees rather than on the true generative grid, which is exactly the centroid-merging effect visible in Table 7 at low SNR.

References

- [1] C. Huang, A. F. Molisch, R. He, R. Wang, P. Tang, B. Ai, and Z. Zhong, “Machine Learning-Enabled LOS/NLOS Identification for MIMO Systems in Dynamic Environments,” *IEEE Transactions on Wireless Communications*, vol. 19, no. 6, pp. 3643–3657, Jun. 2020. doi: 10.1109/TWC.2020.2967726.
- [2] Dhavala.H, “IMT2023579-COM-837-ML-for-Wireless-Communication,” GitHub repository. [Online]. Available: <https://github.com/DHardhik/IMT2023579-COM-837-ML-for-Wireless-Communication>